One Last Run

Estimated time: 30-40 minutes
Laboratory Goals: Recall and distinguish core OOP concepts (abstraction, inheritance, composition) by applying them to concrete design scenarios, evaluate design decisions against SOLID principles, apply exception handling & serialization knowledge and finally, give a proper reasoning behind each selected answer.

Story Background

It's 10:01 PM. The lights in the research lab are still on. A few hours ago, ARIS - the Autonomous Research Intelligent System went silent. ARIS is not just any system. She runs the university's entire research grid from climate simulations to automated grading systems for every course in the faculty.

So, what exactly happened? A catastrophic design failure buried deep in her codebase. That's what happened. Why? A single developer was working alone, under deadline pressure so they ignored eleven weeks of object-oriented principles and wrote everything as one 4k line God Class with no interfaces, no abstraction, and exception handling that consisted entirely of catch (Exception e) {}. When a routine file I/O operation failed at 8:05, the exception was silently swallowed. ARIS never recovered.

The senior engineering team is currently unreachable. They're either on a conference in Oslo or on a vacation in Miami.

You are all that's left.

The Chief Architect has left behind a diagnostic questionnaire (weird but why not) - 20 questions that probe every layer of the system. Each question represents a real decision point in ARIS's codebase. You need to answer as many questions correctly as possible, explain your reasoning and the protocol to restore ARIS will have enough signal to rebuild her. You skip the reasoning and the protocol fails silently.

ARIS is counting on you. So is your 1 point for task 1. Your final chance.

Mission: Select the correct answer and give a brief reasoning.

OOP Foundations & SOLID

Q1	Inheritance & interfaces	Week 6 - 7

A Dog and a Robot both need a move() method. They share no fields and no implementation. What should they extend or implement?

- ☐ An abstract class Mover
- ☐ A Movable interface
- ☐ A concrete class Mover
- ☐ Nothing. Just duplicate the method

My reasoning:

Q2	SOLID	Week 9

What's wrong with this code? Which SOLID principle is it violating?

```
class Bird { public void fly() { ... } }
class Penguin extends Bird {
    public void fly() { throw new UnsupportedOperationException(); }
}
```

- ☐ Open/Closed Principle
- ☐ Single Responsibility Principle
- ☐ Liskov Substitution Principle
- ☐ Nothing. This is valid Java.

My reasoning:

Q3	Abstraction	Week 6 - 7

You have 6 classes. All share 2 fields and a helper method, but each has a completely different core behavior. Best structure?

- ☐ Abstract class with shared fields + abstract method for core behavior
- ☐ Interface only. Keep it flexible
- ☐ One concrete class with a type flag and if/else
- ☐ 6 unrelated classes to avoid coupling

My reasoning:

Q4	Composition	Week 6 - 7

A Car class needs an Engine. Which is the better design?

```
// Option A
class Car extends Engine { ... }

// Option B
class Car {
    private Engine engine;
    public Car(Engine engine) { this.engine = engine; }
}
```

- ☐ Option A
- ☐ Option B
- ☐ Both are the same
- ☐ Option A if Engine is abstract, B otherwise

My reasoning:

Q5	SOLID	Week 9

A Printer class implements MachineOperations which has print(), scan(), and fax(). The printer can't fax. What does it do?

- ☐ Nothing. It just doesn't override fax()
- ☐ It extends the interface and adds a default
- ☐ It's forced to implement fax(), probably throwing an exception which is ISP violation
- ☐ It implements only the methods it needs

My reasoning:

Generics, Lambdas & Iterators

Q6	Lambdas	Week 11

Which line causes a compile error?

```
List<Integer> ints = new ArrayList<>();
ints.add(42);

List<Number> nums = ints;        // line A
List<? extends Number> view = ints; // line B
```

- ☐ Line A because generics are not covariant
- ☐ Line B because wildcards are not allowed here
- ☐ Both compile fine
- ☐ Neither because Java auto-widens numeric types

My reasoning:

Q7	Lambdas	Week 11

You want to sort a list of names by length, shortest first. Which lambda is correct?

- ☐ (a, b) -> a.length()
- ☐ (a, b) -> a.length() - b.length()
- ☐ a -> a.length()
- ☐ (a, b) -> a.compareTo(b)

My reasoning:

Q8	Iterator	Week 11

Your class implements Iterable<T>. What is the minimum you must provide?

- ☐ A get(int index) method
- ☐ Both hasNext() and next() directly on the class
- ☐ An iterator() method that returns an Iterator<T>
- ☐ Extending ArrayList

My reasoning:

Q9	Generics + lambdas	Week 11

What does this method accept?

```
public <T> void process(List<T> items, Predicate<T> condition) {
    for (T item : items)
        if (condition.test(item)) System.out.println(item);
}
```

- ☐ Any list, and condition can be a lambda
- ☐ Any list, but condition must be a named class
- ☐ Only lists of strings
- ☐ This doesn't compile. It can't mix generics and lambdas

My reasoning:

Q10	Big picture / Design decision	Week 1-11

A junior dev says: "I just use ArrayList everywhere and write logic directly in main(). It works, so why bother with all this OOP stuff?" What's the strongest counter-argument?

- ☐ It won't compile on larger projects
- ☐ It's against Java conventions
- ☐ It works until requirements change. OOP makes change cheap
- ☐ It's slower at runtime

My reasoning:

Exceptions

Q11	Exceptions	Week 10

This code compiles and runs. What actually prints?

```
try {
    System.out.println("A");
    if (true) throw new RuntimeException();
    System.out.println("B");
} catch (RuntimeException e) {
    System.out.println("C");
} finally {
    System.out.println("D");
}
```

- ☐ A, B, D
- ☐ A, C
- ☐ A, D
- ☐ A, C, D

My reasoning:

Q12	Exceptions	Week 10

You're writing a method that reads a user's age from input. The input is "abc". What's the right thing to do?

- ☐ Catch the exception and throw a custom
- ☐ Return -1 as a sentinel value for invalid input
- ☐ Let the NumberFormatException crash the app. It's the user's fault.
- ☐ Use an if-statement to check every character manually instead

My reasoning:

Q13	Checked vs unchecked	Week 10

Your method calls a file reader that throws IOException. You don't handle it. What must you do?

- ☐ Nothing. Java handles I/O exceptions automatically
- ☐ Declare throws IOException in the method signature
- ☐ Wrap everything in a try/catch for Exception
- ☐ Only handle it if the file might actually be missing

My reasoning:

Q14	File I/O	Week 4

What is the advantage of wrapping a FileReader in a BufferedReader?

- ☐ It automatically closes the file when done
- ☐ It reduces disk reads by reading chunks into memory at once
- ☐ It lets you read binary files as well as text
- ☐ It handles encoding differences between operating systems

My reasoning:

Q15	File I/O	Week 4

You try to serialize a Student object but get a NotSerializableException. What's most likely missing?

```
class Student {
    String name;
    int grade;
}
```

- ☐ A toString() method
- ☐ A no-argument constructor
- ☐ implements Serializable
- ☐ The fields must be public

My reasoning:

Q16	Serialization	Week 4

A User class has a password field. You serialize it to a file. What should you do to prevent the password from being saved?

- ☐ Make the field private
- ☐ Mark the field transient
- ☐ Set it to null before serializing
- ☐ Serialize only the fields you want manually

My reasoning:

Android Studio

Q17	Android Studio, ViewBinding	Week 8-11

Why is ViewBinding preferred over findViewById()?

- ☐ It's faster at runtime because it skips the XML parser
- ☐ It lets you define views directly in Java without any XML
- ☐ It automatically updates the UI when data changes
- ☐ It's null-safe and type-safe — no casting, no NullPointerException from wrong IDs

My reasoning:

Q18	Android Studio, RecyclerView	Week 8-11

Why is ViewBinding preferred over findViewById()?

- ☐ RecyclerView reuses view holders — only visible items exist in memory
- ☐ RecyclerView compresses the view data into fewer objects
- ☐ RecyclerView loads all views faster using multithreading
- ☐ ScrollView can't handle more than 100 child views

My reasoning:

Q19	Adapter pattern, Android Studio	Week 8-11

In a RecyclerView Adapter, what is the role of onBindViewHolder()?

- ☐ Creates a new ViewHolder object for each item
- ☐ Fills a recycled view with data for a specific position
- ☐ Decides how many columns the list has
- ☐ Inflates the XML layout for each row

My reasoning:

Q20	RecyclerView, Android Studio	Week 8-11

A RecyclerView Adapter extends RecyclerView.Adapter<YourViewHolder>. Which OOP concept does this most directly demonstrate?

- ☐ Generics + inheritance
- ☐ Composition
- ☐ Encapsulation
- ☐ Interfaces

My reasoning: